Atticus Functional Requirements Review

This document captures a comprehensive review of the Atticus repository with a focus on extending functionality. The central recommendation is to evolve Atticus from a voice-triggered batch processor into a durable asynchronous personal operations system. The emphasis should be on control, continuity, correction, approvals, retrieval, and long-term project context.

Priority 1 – Command Lifecycle Controls

Add support for command cancellation, amendments, retries, and status inquiries. Commands should move through explicit states such as received, transcribed, queued, running, awaiting approval, completed, failed, cancelled, or superseded. Users should be able to cancel or modify recent requests and retry failed work from voice commands.

Priority 2 – Controlled External Actions

Introduce a risk-based action framework. Classify actions as read-only, reversible writes, externally visible writes, and destructive actions. Support approval queues, drafts, previews, dry-run modes, and idempotent execution to prevent duplicate actions.

Priority 3 – Persistent Context and Projects

Create named project contexts that maintain related tasks, artifacts, repositories, documents, contacts, instructions, and preferences. Allow commands to continue, revise, or extend previous work. Support reusable preferences and bounded context packs.

Priority 4 – Multi-Step Command Composition

Support multiple tasks in a single recording. Allow sequential, parallel, and conditional execution. Resolve references such as 'that report' or 'the second option' before splitting work.

Priority 5 – Capture Modes

Provide explicit modes for notes, ideas, meetings, and dictation. Meeting mode should extract action items, decisions, and follow-ups. Idea mode should generate structured idea cards with tags and next steps.

Priority 6 – Search and Retrieval

Enable search across transcripts, reports, projects, and generated artifacts. Allow users to ask questions about prior work and receive answers grounded in existing artifacts. Generate daily and weekly summaries and support artifact collections.

Priority 7 – Rich Output Contracts

Retain HTML as the canonical output format while supporting PDF, Markdown, DOCX, CSV, JSON, images, source code, and calendar files. Introduce artifact versioning and standardized report sections.

Priority 8 – Skill Governance

Expand skill metadata to include permissions, risk classifications, required credentials, runtime expectations, costs, and output types. Add skill health checks and permission scoping.

Priority 9 – Cost and Resource Controls

Support budgets, runtime limits, workload priorities, and detailed cost accounting for transcription, models, APIs, and integrations.

Priority 10 – Notification Improvements

Improve notification routing, escalation policies, quiet hours, and completion summaries so users can quickly determine outcomes.

Recommended New Skills

Project Capture Revise Artifact Status and Recovery Meeting Actioner Repository Work Document Transform Watch Condition Contact Resolver

Command Control Layer

Introduce a central command model: Recording -> Command -> Tasks -> Project Context -> Risk Classification -> Approval State -> External Actions -> Artifact Versions Relationships: amends, retries, cancels, continues, references, derived_from, supersedes This single architectural addition unlocks many advanced features.

Implementation Order

1. Status, cancel, retry, amend 2. Approval queue and risk tiers 3. Named projects and continuation 4. Multi-task recordings 5. Search across artifacts 6. Meeting mode and project capture 7. Conditional monitoring 8. Additional integrations 9. Multi-format output 10. Cost and notification policies

Guiding Principle

Do not merely increase the number of things Atticus can do. Increase the operator's ability to control, continue, correct, audit, and retrieve what Atticus does.